Technical Audit Report

One Degree Advisory — Laravel Website

Performance · Security · Maintainability · Code Quality

Date: 23 July 2026 | **Scope:** Full codebase

Mode: Review only — no code was modified

Test baseline: 58 automated tests pass (365 assertions)

Method: 5 parallel specialist reviews (backend, security, frontend, dead-code, dependencies) + manual verification of every critical finding

Dimension	Score
Overall health	6 / 10
Performance	5 / 10
Security	4 / 10
Maintainability	7 / 10

Contents

How to read severity labels: CRITICAL exploitable now / breaks things HIGH serious, fix soon MEDIUM should fix LOW minor / hardening CLEAN checked and OK

1. Project Overview

Framework & technologies

Core framework	Laravel `13.11.2` (server-rendered Blade templates; no single-page-app front end)
Language / runtime	PHP — declared `^8.3`, but the dependency lock actually requires `PHP ≥ 8.4` (see §8 — important before next deploy)
Database	SQLite in development; relational tables for the CRM. Marketing/content data is stored as **JSON files** in `storage/app/*.json`, edited through a custom admin CMS
Payments	Razorpay (order → confirm → webhook), prices decided on the server
AI	Groq / Ollama, used by the Visa Mock Interview tool
PDF / Excel	dompdf (HTML→PDF), FPDI/tFPDF (merge PDF pages), smalot/pdfparser (read PDF text), a hand-written spreadsheet reader/writer
Front-end assets	Hand-authored `public/styles.css`, `script.js`, `stripe-nav.*` loaded directly. The Vite/Tailwind build tool is installed but **unused at runtime**

Main modules

- **Public marketing pages** — home, about, careers, contact, privacy, services (test-prep / admissions / student-services), courses (UG / PG / LLB / MBA / doctoral), country guides, MBBS country guides, blog, and CMS-built "brief" pages.
- **Student-Hub tools** — Career Library explorer, Student Profiler wizard, Visa page, AI Visa Mock Interview, Loan & Accommodation, Statement of Purpose.

- **Admin CMS** (`/admin`, one shared password) — blog, about, home-hero, brief-page builder, notice bar, test-prep compare, career library, destinations menu, and a country-data sync tool that runs Python scrapers.
- **Lead CRM** (`/crm`, one-time-passcode email login, per-user roles) — leads, enrollments, subscribers, website submissions, team management, audit log, and a PDF shortlisting tool.
- **Payments** — Razorpay, server-priced.

Application architecture

Thin controllers → "store/content" helper classes that wrap the JSON files → Blade templates. The CRM uses proper database models with a rich, well-indexed schema. Two custom access gates: `CmsAuth` (admin) and `CrmAuth` (CRM one-time-passcode). **No shared-instance (singleton) wiring exists** — every time the code asks for a store it builds a brand-new one and re-reads the file.

Important user flows

- **Visitor opens a page** → each page re-reads roughly 0.9 MB of navigation JSON from disk.
- **Lead capture** (contact / careers / tool forms) → a transactional database write, then emails sent **synchronously** (visitor waits).
- **Payment** → server decides the price → Razorpay → signature-verified confirm + webhook.
- **Admin edits content** through the CMS, which rewrites JSON files.
- **CRM user** logs in with an emailed passcode → dashboard.

Build & deployment

No continuous-integration pipeline. Deployment is a server-side git pull (`pull on live` / `pull on test` over SSH). CMS JSON data is deliberately not in git and is restored from a server backup. The `.gitignore` is correct (env file, database, CMS JSON, vendor, node_modules all excluded). `config:cache` is practised on the server per the README.

2. Critical Issues

CRITICAL **CRIT-1 — Hardcoded default admin & super-admin passwords, committed to git**

Location: `config/site.php:18` — `'cms_password' => env('CMS_PASSWORD', 'onedegree')`; `config/site.php:23` — `'super_admin_password' => env('SUPER_ADMIN_PASSWORD', 'infolith@123')`. *(Independently verified by reading the file.)*

Problem: Both passwords have a fallback value written directly into the source code. If the production environment file does not set `CMS_PASSWORD` / `SUPER_ADMIN_PASSWORD`, the entire `/admin` control panel ships with these publicly-known, committed passwords. The whole admin surface is protected by a single shared password — there are no individual admin accounts. The super-admin password unlocks every CMS page, including the payment-publishing gate, all four remote-URL import tools, and the site-wide notice bar.

Impact: Anyone with repository access (or who guesses the well-known value) can take full control of the CMS — deface every page, inject scripts that run on all visitors, tamper with payment blocks, and trigger server-side requests.

Fix: Remove the literal fallbacks so the app fails closed if the value is unset; set strong values in each production environment file and rotate the current ones. Longer term, move to per-user hashed admin accounts.

Risk of fix: Medium (must set the environment value first or admins get locked out). **Auto-apply:** No.

CRITICAL **CRIT-2 — Reflected script-injection (XSS) on the public Career Library page**

Location: `CareerLibraryController.php:30` passes the raw `?error=` web-address value to the page → `career-library/index.blade.php:477` places it into `<p>${error}</p>` → line 482 injects that into the page via `innerHTML`. *(Independently verified end-to-end.)*

Problem: The value is safely encoded as a text string when written into the script, but it is then rebuilt into live HTML on the page, so tags survive. Opening `/global-career-library?error=<img src=x onerror=...>` runs attacker code in the visitor's browser — unauthenticated, on page load.

Impact: A crafted link shared with your visitors can steal their session/cookies or perform actions as them. This is a public URL, so it is trivially weaponizable.

Fix: Show the message using `textContent`, or HTML-escape the value before building the HTML.

Risk of fix: Low. **Auto-apply:** Yes.

CRITICAL **CRIT-3 — Admin login has no rate limiting**

Location: `routes/web.php:200` — the admin login POST has no `throttle` guard, unlike every other login route on the site. *(Verified.)*

Problem: An attacker can submit unlimited password guesses with no lock-out. Combined with CRIT-1's single shared password, this makes online password-guessing practical. (The password comparison itself is timing-safe and the session is regenerated on success — the only gap is rate limiting.)

Fix: Add `->middleware('throttle:5,10')` to the login POST (matching the CRM passcode routes).

Risk of fix: Low. **Auto-apply:** Yes.

CRITICAL **CRIT-4 — Outdated dependencies with published security advisories**

Location: `composer.lock`. An offline `composer audit` reports real advisories, all fixable by a plain `composer update` within the existing version rules:

- `guzzlehttp/guzzle 7.10.3` — 7 medium advisories (proxy downgrade, cookie/header leaks). This HTTP client is on hot paths: Razorpay, Groq, the OTP SMS sender, and the four admin URL-import tools.
- `guzzlehttp/psr7 2.10.1`, `laravel/framework 13.11.2` (signed-URL issue — not currently exploitable here, no signed URLs used), and several `symfony 8.0.x` components.

Fix: Run `composer update` (targets guzzle ≥ 7.15.1, framework 13.21.1, symfony 8.0.13) and re-run the test suite. **Good news:** dompdf 3.1.5 is clean — past its old remote-code CVEs.

Risk of fix: Low–medium (minor version bumps). **Auto-apply:** Yes, with a test run.

3. Unused Code and Features

Every item was traced against all reference paths: static and dynamically-built template names, includes/extends, PDF template loads, route-name vs. usage, config reads, tests, JavaScript/CSS URLs, and the CMS JSON data files.

3.1 Confirmed unused — zero readers, safe to remove

Item	Path	Why it is unused	Action
Old brief editor view	`resources/views/admin/brief/edit.blade.php`	The controller's `edit()` now only redirects to the studio; nothing renders this view. It also holds the single broken route reference.	Delete
Old editor block partial	`.../admin/brief/_block.blade.php`	Included only by the dead edit view above.	Delete
Old editor preview partial	`.../admin/brief/_preview.blade.php`	Zero references anywhere.	Delete
Dotted world-map partial	`.../partials/codepen-dotted-world-map.blade.php`	Never included; its matching CSS block is dead too.	Delete (confirm MBBS page doesn't inline it)
Sample PDF (in git)	`output/pdf/sample-shortlisting-merged.pdf`	A generated test artifact, referenced nowhere.	Remove from git; ignore `/output/`
Python bytecode (in git)	`scripts/__pycache__/*.pyc`	Compiled cache committed by mistake.	Remove from git; ignore
Empty folders	`.agents/, tmp/, .codex/`	Completely empty.	Delete
Unused config keys	`config/site.php → tagline, notice`	Never read (only `notices`, plural, is used).	Remove keys
Local debug JSON	`storage/app/student-profiler-{lock-check,measure}.json`	No code reads or writes them (old tooling leftovers).	Delete locally

3.2 Probably unused — no live path, but worth a glance

Item	Why it appears unused	Risk	Action
npm / Vite / Tailwind toolchain	No template loads its build output; the entry files are empty stubs; the site loads hand-written CSS/JS directly.	Low	Remove toolchain (after the Tailwind-source check in 3.3)
Default login scaffolding: `User` model, `UserFactory`, `DatabaseSeeder`	The app uses its own custom gates; standard Laravel login is never used. Only the auth config still points at the User model.	Low	Keep (conventional) or remove + repoint config

MBBS scraper `scripts/mbbs_bookmyuniversity.py` + its data file	The PHP code only ever runs the other two scrapers.	Low	Confirm with team, then delete
Single-answer assessment endpoint `visa-mock.assess`	The interview page only calls the batch endpoint; only tests hit the single one.	Med-low (stale cached pages)	Keep 1 release, then remove with its test
Orphan endpoints `admin.pages.import`, `crm.website.destroy`	No button, script, or test references them.	Low	Re-add UI or remove
Skeleton mail-service configs (`postmark, resend, ses, slack`)	Never read; the site uses SMTP mailers.	None	Optional cleanup
Unused env keys (`AWS_*, REDIS_*, MEMCACHED_HOST, BROADCAST_CONNECTION`)	No matching feature is used anywhere.	None	Trim from the example env for clarity

3.3 Requires manual verification — do NOT delete without confirming

- **CRM website-lead export buttons.** The CSV/Excel export routes still exist and are covered by tests, but **no button in the interface links to them** — this looks like a feature accidentally dropped during the CRM redesign (a *regression to fix*, not dead code).
- **Tailwind source file.** The compiled `public/career-library/output.css` was built by an older tool not present in the current setup. Confirm what regenerates it before removing the build toolchain.
- **Evaluator dead branches** inside `ProfileReportBuilder.php`. The `/evaluate-my-profile` feature is fully deleted; these branches are now unreachable but sit next to live profiler logic. Prune only with the mail-test suite green.
- **Legacy import chain** — `profile-submissions.json` + `newsletter-subscribers.json` are read only by the one-off legacy import command. Retire only after confirming every environment has run the import.
- **Assets referenced only from CMS JSON** (e.g. some test-prep course images, hero photos). The production CMS data is the source of truth — check the live JSON before deleting anything under `public/assets`.

3.4 Must NOT be removed — looks dead to a simple search, but is alive

All `partials/brief/blocks/*` (29 files) and `partials/about/*` (6) are included dynamically by content type. The studio partials, the two PDF templates (loaded via `Pdf::loadView`), the `q1–q10.mp3` audio (built as dynamic URLs in JavaScript), externally-hit routes (payment webhook, sitemap, robots, brief pages, career-library detail), the whole Student Profiler module, the two active Python scrapers, all PDF/zip composer packages, and `robots-noindex.txt` — all confirmed live.

3.5 Broken reference

Exactly one, and it is harmless today: `admin/brief/edit.blade.php:211` calls a route `admin.pages.preview` that does not exist. It would crash if reached, but the view is never rendered. Deleting the dead editor (3.1) removes it.

4. Unused Dependencies

All **production** composer packages are genuinely used. The three PDF packages are chained, not redundant: dompdf renders a new page → FPDI merges it after the original pages → smalot reads the text — all in the CRM shortlisting tool.

Package	Version	Used for	Required?	Action
`barryvdh/laravel-dompdf`	3.1.2 (dompdf 3.1.5)	Profiler PDF report; CRM shortlist page (HTML→PDF)	Yes	Keep
`setasign/fpdi`	2.6.8	Merge uploaded PDF pages	Yes	Keep
`setasign/tfpdf`	1.33	UTF-8 base for the above	Yes	Keep
`smalot/pdfparser`	2.12.5	Read student name from page 1 of an uploaded PDF	Yes	Keep
`nelexa/zip`	4.0.2	Read uploaded Excel files (no PHP zip extension needed, by design)	Yes	Keep
`laravel/tinker`	3.0.2	Command-line REPL only; no code references	Optional in production	Consider moving to dev-only
`npm devDependencies`	—	vite, tailwindcss, @tailwindcss/vite, laravel-vite-plugin, concurrently	**No** — builds 3-byte stub files nothing loads	Remove toolchain (after 3.3 check); 54 MB of local node_modules

5. Database Performance Review

STRENGTH The database design is the strongest part of the codebase. Indexing is comprehensive (the leads table alone has 12 indexes, payments 8, all CRM tables covered), the data-loading uses correct "eager loading" so there are **no repeated-query (N+1) problems**, and current data volumes are tiny.

ID	Sev	Finding	Location / evidence	Fix
DB-1	HIGH	SQLite runs without "WAL" mode or a busy-timeout, so one writer blocks everyone and busy moments fail with "database is locked".	`config/database.php:35-45 (all null)`	Set `journal_mode=wal`, `busy_timeout=5000`, `synchronous=normal`
DB-2	MED	A `LOWER(email)` comparison on every lead capture defeats the email index (full-table scan). Emails are already stored lowercased, so it is unnecessary.	`WebsiteLeadManager.php:193,200,219; CrmLeadController.php:38,86,252`	Compare directly on `email`; add an expression index if kept

DB-3	MED	CSV import inserts each row as a separate committed transaction — a large file takes minutes.	`CrmLeadController.php:246-275`	Wrap the loop in one database transaction
DB-4	MED	The leads table has no index on the column it is sorted by (`updated_at`) despite the default dashboard sort.	Migration `..._create_crm_tables`	Add `updated_at` / `created_at` indexes
DB-5	MED	The dashboard chart pulls every lead row for 6 months and groups them in PHP instead of asking the database to group.	`CrmDashboardController.php:287-291`	Use a `GROUP BY` aggregate query
DB-6	LOW	CSV/Excel exports load the whole result set into memory before streaming.	4 export methods	Stream with `cursor()` / `chunkById`

No table or column deletions are recommended. The standard Laravel tables (users, cache, jobs, sessions) are currently idle because of the file/sync settings, but they must stay — they activate the moment those settings change.

6. Frontend Performance Review

The `public/assets` folder is 16 MB (photos are 10 MB of it). The main style sheet `styles.css` is **577 KB / 25,579 lines and blocks every page from rendering**, with no minification. Your own JavaScript is correctly deferred; the real problems are third-party libraries, large images, dead code, and code embedded inside each page's HTML.

Biggest assets

Asset	Size	Where used
`assets/test-prep/IELTS page_1.png`	2.6 MB	**Nowhere** — zero references
`assets/heroes/*.jpg`	5.7 MB (19 files; worst 664 KB)	Blog + country-data JSON (still in use)
`styles.css`	577 KB (101 KB compressed)	Every page, render-blocking
`assets/sop/game-bg.jpg`	532 KB	Statement-of-Purpose page
`assets/test-prep/inner-hero-bg-2.png`	400 KB	**Nowhere** (only the webp version is used)
Lucide icons (remote, unpkg)	~90 KB + a redirect	Every page; unpinned "latest"
Home hero (remote Unsplash)	~350–500 KB, 2200px wide	Homepage main image, not preloaded
jQuery (remote)	~30 KB compressed	Career Library page, loaded blocking

Findings

ID	Sev	Finding	Fix
FE-1	HIGH	Lucide icon library loaded as "latest" from a CDN on every page — extra redirect + ~90 KB.	Pin a version; self-host/subset later
FE-2	HIGH	The homepage's main image is remote and not preloaded — the biggest homepage slowness.	Add a preload hint (address is known on the server)
FE-3	HIGH	Country-page hero images are up to 592 KB and not preloaded.	Preload + recompress the 5 worst
FE-4	MED	About a quarter of styles.css (~6,900 lines) is dead leftovers from removed features (old menus, old page designs, three old MBBS layouts).	Delete verified sections + add minification
FE-5	MED	~32 KB of script.js is dead code (an unused search box, an unreachable country-data fetch, an old menu controller).	Delete (search-verified)
FE-6	MED	41 country-flag images load on *every* page even though the menu is hidden.	Add lazy-loading (2-attribute change)
FE-7	MED	jQuery loaded (blocking) on the Career Library page for just 3 tiny actions.	Replace with ~6 lines of plain JavaScript
FE-8	MED	The Visa Mock Interview page embeds ~176 KB of CSS+JS inside its HTML (re-downloaded every visit).	Move to cacheable external files
FE-9	MED	A 118 KB PDF library loads on every Visa Mock visit though it is only used at the very end.	Load it only when needed
FE-10	MED	Animation library (GSAP) loaded blocking on the Statement-of-Purpose page.	Add "defer"

FE-11	MED	Blog serves 300–660 KB .jpg heroes while smaller .webp versions already exist.	Point the CMS data at the .webp files (server-side)
FE-12	LOW	100 of 110 images have no width/height (causes layout shift); no responsive images anywhere.	Add dimensions to the repeated card layouts first
FE-13	LOW	Up to 8–10 font families load on some tool pages.	Trim overlapping fonts
FE-14	LOW	A timer runs every 0.45s forcing layout recalculation on long pages.	Rely on the existing scroll observer alone
FE-15	MED	Razorpay checkout script loads blocking when a payment block is on the page.	Load on first "pay" click

QUICK WIN Doing FE-1, 2, 3, 5, 6, 7, 10 together saves roughly **150–200 KB and 40+ network requests per page**, plus the biggest homepage and country-page speed gains — all with **zero visual change**.

7. Backend Performance Review

The recurring theme: **there is no caching anywhere**, and no shared instances — so every content file is read and decoded from disk on every request, often several times.

ID	Sev	Finding	Location	Fix
BE-1	HIGH	The 11.5 MB Career Library file is fully read and processed on every request (~46–110 ms, up to 66 MB memory) — even on the small lead-capture form submissions that only need one setting.	`CareerLibraryStore.php:66-82; Controller:100,179`	Remember it in cache keyed on file-modified-time; share one instance
BE-2	HIGH	Every page decodes ~0.9 MB of navigation data via fresh instances; the notice bar is decoded twice; country pages decode the same 632 KB file three times.	`header-stripe.blade, app.blade, notice-bar.blade, StudyLocationContent.php`	Share instances + remember results in cache
BE-3	HIGH	All 10 email sends happen synchronously while the user waits. The profiler even renders a PDF and then sends 2 emails before responding. The Razorpay webhook sends 2 emails inline (can exceed Razorpay's timeout and trigger repeat calls).	`PageController, PaymentController, ProfileReportNotifier`	Send email in the background (the queue table already exists), or at least after the response for the webhook
BE-4	HIGH	The Visa Mock AI endpoints can hold a server worker busy for ~7–12 minutes (the AI timeouts stack up one after another).	`VisaMockAssessor.php:280-292`	Cap the local-AI timeout, or move to a background job + polling
BE-5	MED	The "page not found" fallback reads the brief-pages file on every 404, and can even rewrite the file on a simple page view.	`BriefPageController; BriefPageStore.php:42`	Cache; never write on a GET request
BE-6	MED	9 content-file writers save without a file lock — two simultaneous saves can corrupt the JSON.	9 store classes	Add a write lock (one line each)
BE-7	MED	The CRM dashboard runs every tab's queries on every load, including two unbounded scans of full tables.	`CrmDashboardController.php:20-172`	Only run the queries for the tab being viewed
BE-8	MED	The sitemap decodes ~1.1 MB per crawler visit; the payment resolver re-reads the brief-pages file on every order.	`SeoController; PaymentBlockResolver`	Cache both
BE-9	LOW	No scheduled maintenance — old passcodes, stale payment attempts, and uploaded resumes are never cleaned up.	`routes/console.php`	Add a cleanup schedule

CLEAN Verified good: no repeated-query problems, the login gates use at most one indexed query, payment look-ups hit unique indexes, uploads are streamed to disk (not held in memory), and the profiler page load is light.

8. Security and Configuration Review

The four most serious items are in Section 2. The rest:

Medium

ID	Finding	Fix
SEC-M1	**Server-side request forgery (SSRF)** in all four admin "import from URL" tools: they check only that the address starts with http(s) and then fetch it (following redirects). An admin-supplied address could reach internal-only services or cloud metadata.	Block private/internal addresses (including after redirects); disable auto-redirect
SEC-M2	**Stored, site-wide script injection** via the notice-bar text: it is saved without cleaning and printed as raw HTML on *every* page. The in-code comment claiming it is sanitized is false.	Clean the HTML on save (allow only a few safe tags), or store plain text + structured links
SEC-M3	**Attribute injection (XSS)** via the career name in the web address (same root cause as CRIT-2), active when detail pages are on.	Escape the value before use
SEC-M4	**SVG upload → stored script injection.** The image rule allows .svg files, which are served from a web-accessible folder and can contain scripts. The claimed file type is trusted, not verified.	Drop SVG from the allow-list, or serve uploads as downloads only

Low / Info

- **SEC-L1** — The structured-data (JSON-LD) block isn't tag-escaped; CMS country text containing a closing script tag could break out. Add stricter encoding.
- **SEC-L2** — The "remember me" cookie is a fixed, role-shared token (unforgeable, but not revocable per device).
- **SEC-L3** — The secure-cookie flag isn't set; enable it in production.
- **SEC-L4** — No security response headers anywhere (no Content-Security-Policy, no clickjacking protection). A CSP would blunt CRIT-2, M2, and M3.
- **SEC-L5** — The CRM passcode request reveals whether a phone number is registered.
- **SEC-L6** — Notice-bar links allow the `javascript:` scheme.
- **SEC-L7** — Passcodes are written to the log when debug logging is on (off by default).

Configuration

ID	Finding	Fix
CFG-1 HIGH	Real CRM super-admin **personal data** (names, phone numbers, emails) is committed as default values in `config/crm.php` and the example env file.	Blank the defaults; keep real values only on the server
CFG-2 MED	Production may be running on file-based sessions/cache and synchronous email — verify the server env. These make busy-time behaviour slower and less reliable.	Move sessions/cache to the database (tables already exist)
CFG-3 MED	Logging is set to "debug" level with no rotation → the log file grows without limit.	Set daily rotation + a higher log level in production

CFG-4 LOW	A Python-path setting is read in a way that stops working once config caching is on.	Move it to a config value
CFG-5 INFO	Route caching is **safe** to enable on this version — recommend turning on route + view caching in the deploy step.	Enable in deploy

PHP version trap (act before next deploy): The project declares PHP 8.3, but the locked dependencies actually require **PHP 8.4**. Installing from the current lock file will fail on a real 8.3 server. Confirm the server's PHP version and align the project accordingly.

VERIFIED CLEAN — genuine strengths

- **Payments:** the price is decided on the server (the customer can't change it); Razorpay signatures are checked with a timing-safe method on both confirm and webhook; repeat/replayed confirmations are handled safely; no card data is logged.
- **CRM permissions:** every action is properly role-checked — no way for a normal user to escalate to admin, and no cross-user data access.
- **CRM passcodes:** 6-digit cryptographically-random codes, stored hashed, 5-minute expiry, 5-attempt limit, single use, session refreshed on login.
- **Databases queries** use safe parameter binding — no SQL-injection found.
- **Secrets** are not committed to git (env file, database, keys all excluded); the careers resume upload is validated and stored privately.

Key versions: Laravel 13.11.2, dompdf 3.1.5 (clean), guzzle 7.10.3 (see CRIT-4), symfony 8.0.x.

9. Recommended Cleanup Plan

Immediate — safe, high-impact, low-risk

Item	Benefit	Difficulty	Risk	Rollback
CRIT-2 escape the ?error= value	Kills a public script-injection	Trivial	Low	git revert
CRIT-3 add login rate-limit	Stops password guessing	Trivial	Low	git revert
CRIT-4 update dependencies	Clears known CVEs	Easy	Low-med	Restore lock file
DB-1 SQLite WAL + timeout	No "database locked" under load	Trivial	Low	git revert
FE-1/2/3/5/6/7/10 quick wins	~150–200 KB + 40 req/page, faster LCP	Easy	Low	git revert
Delete confirmed-dead files (3.1)	Cleaner codebase	Easy	None	git revert
CFG-1 blank the committed personal data	Stops leaking staff PII	Trivial	Low	git revert

Short-Term — needs testing or a product decision

- **CRIT-1** remove password fallbacks (set the server env first).
- **BE-1 / BE-2 / BE-8** add caching + shared instances for the content stores — the **single biggest backend speed win** (removes ~1 MB of decoding from every page and the 11.5 MB Career Library re-read).
- **BE-3** send email in the background.
- **SEC-M1 / M2 / M4** SSRF guard, notice-bar cleaning, drop SVG uploads; **SEC-L1 / L4** add security headers.
- **DB-2 / 3 / 4 / 5** query fixes.
- **FE-4** delete dead CSS + minify; **FE-8 / 9 / 12** extract inline assets, lazy-load the PDF library, add image dimensions.
- Fix the **CRM export button** regression; resolve the **PHP 8.3/8.4** mismatch; enable route/view caching.

Long-Term — architectural

- Move to per-user admin accounts (replace the single shared password).
- Database-backed sessions/cache + a background worker on the server.
- Queue the Visa Mock AI work; load the Razorpay script only on demand.
- Remove or properly connect the unused build toolchain; retire the legacy import chain; add a tuned Content-Security-Policy.

10. Final Summary

Dimension	Score	Why
Overall health	**6 / 10**	Solid structure and passing tests, undercut by weak security defaults and no caching
Performance	**5 / 10**	Excellent database work, but every page decodes ~1 MB of JSON + a 577 KB stylesheet + heavy CDNs; no cache layer

Security	**4 / 10**	Payments/permissions/passcodes are strong, but hardcoded passwords + public script-injection + SSRF + no login limit are serious
Maintainability	**7 / 10**	Clean controllers and tests, dragged down by a 25,000-line hand-served stylesheet, dead code, and a phantom build pipeline

- **Removable code:** ~7,000 lines of dead CSS, ~32 KB of dead JavaScript, ~10 dead/orphan template & config items, ~3.3 MB of confirmed-unused images, and the entire npm toolchain.
- **Removable dependencies:** the 5 npm dev-dependencies (the whole build toolchain); `laravel/tinker` can move to dev-only. All production composer packages are used.
- **Top 5 performance improvements:** (1) cache + share the content stores; (2) update dependencies + enable route/view/config caching; (3) SQLite WAL + busy-timeout; (4) the frontend quick-win bundle; (5) send email in the background.
- **Top 5 cleanup opportunities:** (1) delete ~7,000 lines of dead CSS + minify; (2) delete dead JavaScript blocks; (3) remove the unused build toolchain; (4) delete the confirmed-dead template/config/asset set; (5) remove the committed sample PDF, Python cache, and empty folders from git.
- **Needs your confirmation before removal:** the CRM export buttons (likely a regression), the Tailwind source file, the evaluator dead branches, the legacy import chain, any assets referenced only from live CMS data, and the unused MBBS scraper.

Appendix: Proposed Cleanup Checklist

Nothing will be changed until you approve specific batches. Recommended order: Batch A (security) and Batch C (safe frontend wins) first.

Batch A — Security critical (fast, low-risk)

- A1 · Escape the ?error= script-injection (CRIT-2)
- A2 · Add rate-limit to admin login (CRIT-3)
- A3 · Remove hardcoded password fallbacks (CRIT-1) — needs server env set first
- A4 · Blank the committed personal data in config (CFG-1)
- A5 · Add SSRF protection to the 4 import tools (SEC-M1)
- A6 · Clean notice-bar HTML + drop SVG uploads (SEC-M2 / M4)

Batch B — Dependencies & config (low-risk)

- B1 · Run composer update + full test run (CRIT-4)
- B2 · SQLite WAL + busy-timeout (DB-1)
- B3 · Resolve the PHP 8.3 / 8.4 mismatch
- B4 · Add write locks to the 9 content-file writers (BE-6)

Batch C — Frontend quick wins (zero visual change; ~150–200 KB + 40 req/page)

- C1 · Pin the icon library C2 · Preload home + country hero images C3 · Delete dead JavaScript
- C4 · Lazy-load the flag images C5 · Drop jQuery from Career Library C6 · Defer the animation library

Batch D — Dead code / file removal (low-risk)

- D1 · Delete confirmed-dead templates + config keys (3.1)
- D2 · Remove the sample PDF, Python cache, and empty folders from git
- D3 · Delete confirmed-unused images (the 2.6 MB IELTS png, etc.)

Batch E — Backend caching (highest speed impact; needs testing)

- E1 · Cache + share the content stores (BE-1 / 2 / 8)
- E2 · Send email in the background (BE-3)
- E3 · CRM dashboard tab-gating + database query fixes (DB-2 / 3 / 4 / 5)

Batch F — Larger cleanups (decisions needed)

- F1 · Delete ~7,000 lines of dead CSS + add minification (FE-4)
- F2 · Remove the npm / Vite toolchain
- F3 · Extract the Visa-Mock inline assets (FE-8)
- F4 · Investigate the CRM export-button regression

End of report — generated 23 July 2026 · review only, no code modified